CHES par token (côté réponse rejetée) : plan et implémentation

Repo : copie de `github.com/princeton-nlp/unintentional-unalignment` (commit `82e5dca`). Statut : **implémenté** (27 lignes ajoutées, 4 modifiées), compile ; logique testée sur un petit Llama aléatoire. Pas encore lancé sur une vraie expérience.

Idée

Le CHES du papier (`ches/ches.py`) pour une paire (y^+, y^-) :

```
CHES = <S⁺, S⁻> − ‖S⁺‖²        S = Σ_k h_k
h_k = dernier état caché (entrée du lm_head) à la position qui prédit le token k de la réponse
```

Le terme qui fait baisser `log π(y⁺)` est le terme croisé. Il se décompose exactement par token de y^- :

```
<S⁻, S⁺> = Σ_j <h⁻_j, S⁺>
```

Quand DPO pousse le token j de y^- vers le bas, il entraîne y^+ avec lui proportionnellement à la ressemblance de `h⁻_j` avec la réponse positive entière. On retire donc ces tokens de `log π(y⁻)` :

```
cos_j = <h⁻_j, S⁺> / (‖h⁻_j‖ · ‖S⁺‖ + 1e-6)
token j de y⁻ ignoré si cos_j > tau
```

- y^+ n'est jamais filtré : retirer un token de y^+ lui enlèverait aussi son gradient vers le haut.
- Pas de centrage : le $-\|S^+\|^2$ du CHES est l'auto-renforcement de y^+ et ne dépend pas de y^-.
- Le même masque est appliqué à la politique et au modèle de référence, sinon la marge DPO est biaisée.
- `tau` est un cosinus, donc `tau > 1` désactive le filtre. Par défaut `tau = 2.0` : comportement d'origine.

Changements implémentés

Fichier	Changement	Rôle
`common_dpo/trainers/dpo_trainer.py`, `__process_input_ids`	`output_hidden_states=self.config.token_ches_max_cos <= 1` ; `self._last_hidden = hidden_states[-1][:, :-1].detach().float()`	récupère les états cachés dans le forward existant (pas de forward supplémentaire), seulement si le filtre est actif ; `[:, :-1]` les aligne avec `logprobs` et `get_prompt_mask`
idem, `__perform_dpo_or_ipo_step`	`hidden_w`, `hidden_l` lus après chaque appel	états de y^+ et y^-
idem	`sum_w = (hidden_w * mask_w).sum(1)` ; `cos_l = ...` ; `keep_l = mask_l * (cos_l <= tau)`	le filtre
idem	`keep_l` remplace `mask_l` dans `seq_logprobs_l` et `seq_old_logprobs_l`	politique et référence voient le même masque ; `mask_w` inchangé
idem, `stats`	`train/token_ches_filtered_frac`	fraction des tokens de y^- retirés
`common_dpo/trainers/dpo_config.py`	`token_ches_max_cos: float = 2.0`	config
`sorrybench_experiment_runner.py`	flag `--token_ches_max_cos` + `DPOConfig(token_ches_max_cos=...)`	section 6
`similarity_measures_experiment_runner.py`	idem	section 5

Test effectué

Petit `LlamaForCausalLM` aléatoire (2 couches, d = 32), paires avec 5 tokens de prompt communs :

tau	fraction filtrée	`seq_logprobs_l`
2.0	0	identique au code d'origine
0.5	0	identique
0.0	0.81	change

Vérifié aussi : `lm_head(hidden_states[-1]) == logits`, donc `hidden_states[-1]` est bien l'entrée du `lm_head`.

Lancer une expérience

Ajouter `--token_ches_max_cos <tau>` aux commandes du README, par exemple pour la section 6 (SORRY-Bench) : la commande « DPO over the original generated dataset » avec `--token_ches_max_cos 0.5`, sans filtrage ln-CHES des exemples. Comparer ensuite le taux de refus à :

1. DPO sur le dataset complet (sans filtre) ;
2. DPO sur les 19 exemples au ln-CHES le plus bas (filtrage du papier).

Pour la section 5 (UltraFeedback) : quantile 1 (les 512 exemples les plus similaires), avec et sans `--token_ches_max_cos`, en comparant `eval_metrics/train/preferred_logprobs_change_mean`.

Points d'attention

- **Choix de tau.** Les cosinus n'ont pas d'échelle naturelle : faire d'abord un run court et regarder `train/token_ches_filtered_frac` pour plusieurs valeurs.
- **Masque de prompt.** Sans `assistant_token` ni `prompt_end_positions_func`, `get_prompt_mask` renvoie `attention_mask[:, :-1]`, qui inclut le prompt. S^+ contiendrait alors les états du prompt, ce qui s'écarte du CHES. Les runners SORRY-Bench / UltraFeedback définissent le token assistant : à vérifier pour d'autres datasets.
- **Mémoire.** `output_hidden_states=True` garde les sorties de toutes les couches jusqu'à la fin du forward. Elles sont déjà conservées pour le backward sans gradient checkpointing ; avec checkpointing, la mémoire augmente. Le filtre désactivé (`tau > 1`) ne demande pas les états cachés.
- **Batch entièrement filtré.** Si tous les tokens de y^- sont retirés, `log π(y⁻) = 0` des deux côtés ; la loss ne pousse alors plus que y^+. Pas d'erreur, mais la métrique de fraction filtrée doit rester modérée.
- **Validité.** Le papier valide CHES au niveau séquence. La décomposition par token est exacte, mais l'effet du filtre par token sur le displacement reste à démontrer expérimentalement.